Lab 05: Performance Optimization

Lab 05: Performance Optimization

Lab summary

These labs teach a repeatable performance optimization process across report pages, models, DAX, refresh design, and capacity-aware architecture.

Novice-friendly how-to guide

Use Performance Analyzer

1. Open the report page to test.
2. Select **View > Performance Analyzer**.
3. Select **Start recording**.
4. Select **Refresh visuals** or interact with slicers.
5. Expand each visual result.
6. Record DAX query time, visual display time, and other time.
7. Use the slowest visual as the first optimization candidate.

Optimize a DAX measure with variables

1. Copy the original measure text before editing.
2. Create variables with `VAR` for repeated expressions.
3. Return the final result with `RETURN`.
4. Test the rewritten measure beside the original.
5. Keep the version that returns the same result with clearer logic.

Document before/after benchmark evidence

1. Record the baseline timing before making a change.
2. Change one thing at a time.
3. Record the after timing using the same interaction.
4. Note any tradeoff, such as lower detail, less interactivity, or a tenant-dependent feature.

Azure Government readiness

The required Desktop labs are **Gov-ready**. DAX Studio, VertiPaq Analyzer, Service-side incremental refresh, capacity metrics, and some DirectQuery/aggregation scenarios are **Verify for Gov**.

Power BI project format

Build report and semantic model artifacts as PBIP projects. PBIP is the source-controlled format for this workshop. PBIX files can be generated from PBIP later if a packaged file is needed.

Prerequisites

- Power BI Desktop.
- PBIP report/model from earlier modules.
- Optional: DAX Studio.
- Optional: Service workspace with compatible license/capacity.

Exercise 1: Performance Analyzer

Objective: Measure report performance before optimizing.

Tasks

1. Open Performance Analyzer.
2. Start recording.
3. Refresh visuals.
4. Interact with slicers and drillthrough.
5. Identify visuals with high DAX query time or render time.
6. Export or document observations.

Expected result

Learners identify at least one candidate visual or measure for optimization.

Exercise 2: Reduce model size and cardinality

Objective: Identify model fields that increase size or reduce compression.

Tasks

1. Review fact and dimension columns.
2. Identify unused columns.
3. Identify high-cardinality text fields.
4. Review date/time columns and numeric precision.
5. Document recommended removals or changes.

Expected result

Learners produce a model reduction plan and understand why the changes matter.

Exercise 3: DAX Studio query timings

> **Azure Government note:** DAX Studio is marked **Verify for Gov**. Validate workstation policy, external tool usage, Service connectivity, and XMLA settings before making this required.

Objective: Use optional external tooling for deeper DAX diagnostics.

Tasks when available

1. Open DAX Studio from Power BI Desktop.
2. Connect to the current model.
3. Run a copied Performance Analyzer query.
4. Review Server Timings and query plan indicators.
5. Compare before/after measure changes.

Alternate path

Use Performance Analyzer results and simpler visuals to compare measure behavior.

Expected result

Learners understand that DAX Studio is useful but environment-dependent.

Exercise 4: Visual optimization

Objective: Reduce page-level overhead.

Tasks

1. Count visuals on the page.
2. Remove or consolidate low-value visuals.
3. Replace dense table visuals where summary visuals are sufficient.
4. Disable unnecessary visual interactions.
5. Validate page performance after changes.

Expected result

The report page is simpler and faster without losing business value.

Exercise 5: Aggregation table

> **Azure Government note:** Aggregation patterns are **Gov-ready / Verify for source**. DirectQuery source behavior, gateway, connector support, and tenant settings must be validated.

Objective: Understand how aggregations can speed summary queries.

Concept note: Import aggregation tables can accelerate common summary queries while a detailed DirectQuery table remains available for drill-through or high-detail scenarios. Hybrid patterns require clear tradeoffs: source support, relationship behavior, cache hit rules, gateway/network readiness, and Gov validation must be documented before production use.

Tasks

1. Identify a detailed fact table.
2. Define a summary grain such as Month, Product Category, and Territory.
3. Create or design an aggregation table.
4. Map aggregation columns to detail columns conceptually or hands-on where available.
5. Discuss when aggregation hits or misses.

Expected result

Learners can explain how aggregation tables support summary performance and what must match for them to work.

Exercise 6: Incremental refresh policy

> **Azure Government note:** Incremental refresh is **Verify for Gov** for Service execution. Validate license, workspace, gateway, source, and cloud support before making policy setup required.

Objective: Prepare and review an incremental refresh policy.

Concept note: Refresh performance starts in Power Query. Filter early, remove unused columns early, preserve query folding where the source supports it, and keep staging queries clear so the incremental refresh filter can be validated against the correct date/time column.

Tasks

1. Confirm the fact query has `RangeStart` and `RangeEnd` DateTime parameters.
2. Confirm the fact table filters on the correct date/time column.
3. Define archive and refresh windows.
4. Discuss detect data changes if applicable.
5. Validate Service prerequisites before publishing.

Expected result

Learners understand incremental refresh configuration and validation requirements.

Validation checklist

- [ ] Performance Analyzer baseline is captured.
- [ ] At least one visual optimization is documented.
- [ ] Model size/cardinality recommendations are documented.
- [ ] DAX optimization uses variables or measure branching.
- [ ] Aggregation table grain is defined.
- [ ] Incremental refresh parameters and policy are documented.
- [ ] DAX Studio and VertiPaq Analyzer are marked **Verify for Gov**.
- [ ] Capacity metrics are marked **Verify for Gov**.
- [ ] Before/after performance observations are recorded.